Table of Contents

1. INTRODUCTION 4
2.HISTORY 7
Predecessors 7

..... 8
3.AGILE MANIFESTO 9

4.AGILE SOFTWARE DEVELOPMENT CHARACTERISTICS 13

5.TRADITIONAL SOFTWARE DEVELOPMENT 17
Requirements 17
Architecture and Design 17
Development 17
Testing, delivery, and feedback 17

6.COMPARISON WITH OTHER METHORDS 20

7.AGILE METHORDS 23
8.AGILE SOFTWARE DEVELOPMENT TECHNIQUES 25
SCRUM 25
Agile software development with scrum 27
EXTREME PROGRAMMING 28
Feature Driven Development (FDD) 30
Activities 30
Develop overall model 30
Build feature list 31
Plan by feature 31
Design by feature 31
Build by feature 31
Milestones 32

10.SOFTWARE DEVELOPMENT METHODOLOGIES 34
Software development methodology 34

Waterfall development 35
Prototyping 36
Incremental development 37
Spiral development 37
Rapid application development 39
Measuring agility 40

11.SUITABILITY 42
Agile home ground 43
Plan-driven home ground 43
Formal methods 43
Experience reports 43
12.APPLICATIONS OF AGILLE WEB DEVELOPMENT 46
13.REFERENCES 48

LIST OF FIGURES

Fig : 1.1 Agile Software Development 6
Fig 2.1 Photo - Jeff Sutherland 8
Fig:8.1 Scrum 25
Fig:10.1 Prototyping 34

INTRODUCTION

1. INTRODUCTION

Agile software development is a group of software development methodologies based on iterative and incremental development, where requirements and solutions evolve through collaboration between self-organizing, cross-functional teams. The Agile Manifesto introduced the term in 2001. In February 2001, 17 software developers met at the Snowbird, Utah resort, to discuss lightweight development methods. Most promote development, teamwork, collaboration, and process adaptability throughout the life-cycle of the project.Agile methods break tasks into small increments with minimal planning, and do not directly involve long-term planning. Iterations are short time frames (timeboxes) that typically last from one to four weeks. Each iteration involves a team working through a full software development cycle including planning, requirements analysis, design, coding, unit testing, and acceptance testing when a working product is demonstrated to stakeholders. This minimizes overall risk and allows the project to adapt to changes quickly. Stakeholders produce documentation as required. An iteration may not add enough functionality to warrant a market release, but the goal is to have an available release (with minimal bugs) at the end of each iteration. Multiple iterations may be required to release a product or new features.

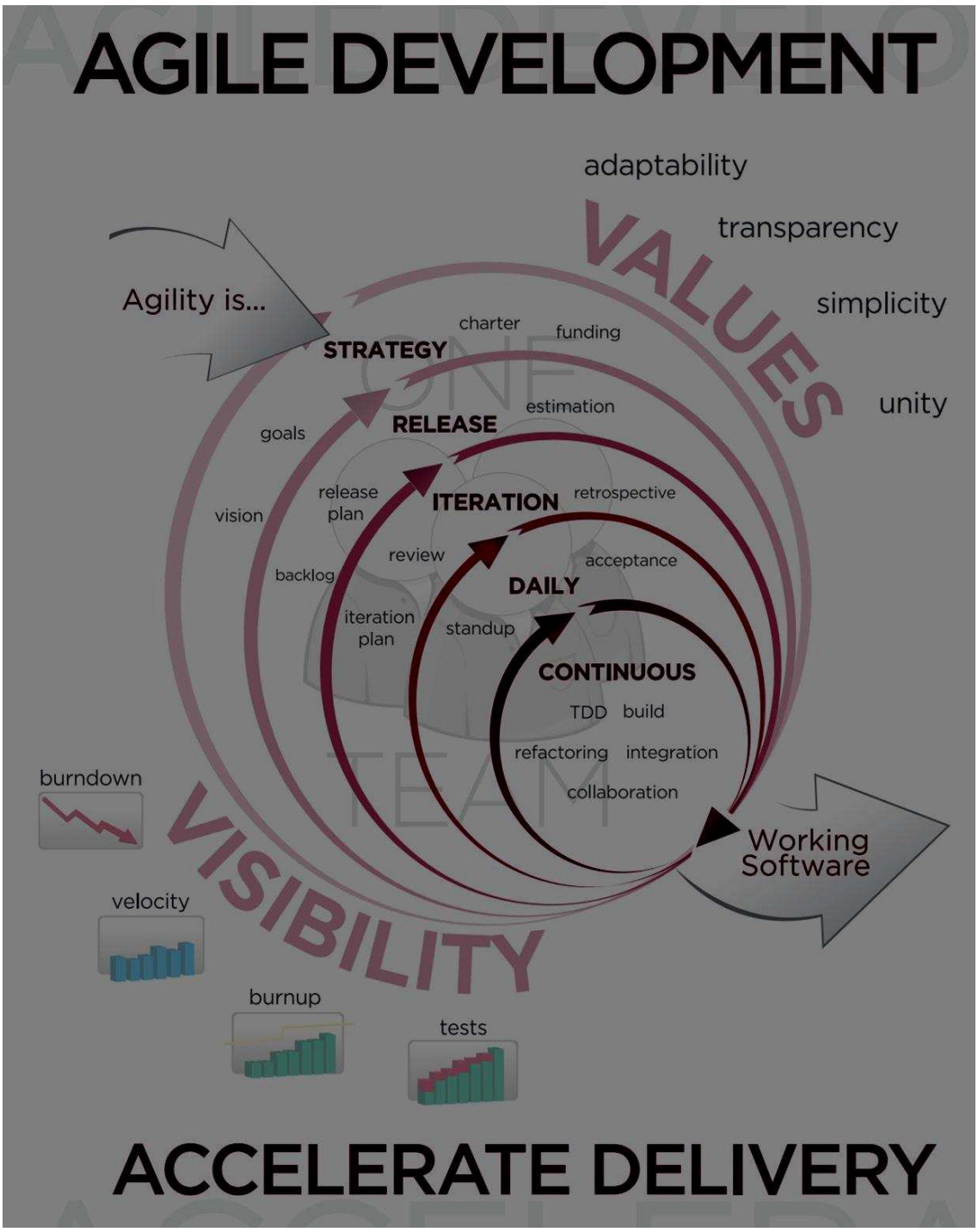

Fig : 1.1 Agile Software Development

HISTORY

2.HISTORY

Predecessors

Jeff Sutherland, one of the developers of the Scrum agile software development process. Incremental software development methods have been traced back to 1957. In 1974, a paper by E. A. Edmonds introduced an adaptive software development process. So-called lightweight software development methods evolved in the mid-1990s as a reaction against heavyweight methods, which were characterized by their critics as a heavily regulated, regimented, micromanaged, waterfall model of development. Proponents of lightweight methods (and now agile methods) contend that they are a return to development practices from early in the history of software development. Early implementations of lightweight methods include Scrum (1995), Crystal Clear, Extreme Programming (1996), Adaptive Software Development, Feature Driven Development, and Dynamic Systems Development Method (DSDM) (1995). These are now typically referred to as agile methodologies, after the Agile Manifesto published in 2001

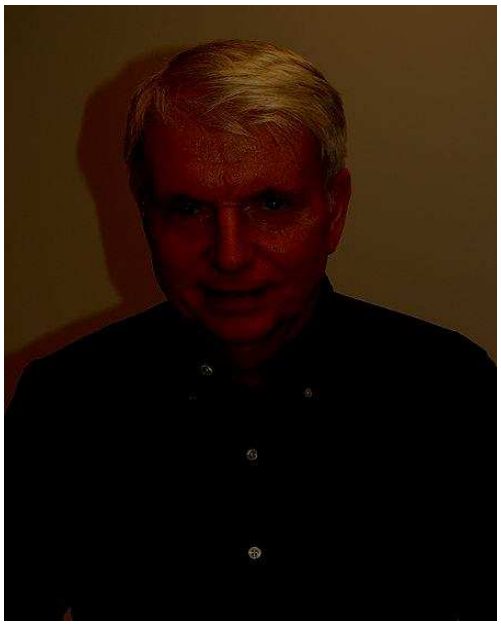

Fig 2.1 Photo - Jeff Sutherland

AGILE MANIFESTO

3.AGILE MANIFESTO

In February 2001, 17 software developers met at the Snowbird, Utah resort, to discuss lightweight development methods. They published the Manifesto for Agile Software Development to define the approach now known as agile software development. Some of the manifesto's authors formed the Agile Alliance, a nonprofit organization that promotes software development according to the manifesto's principles.

Agile Manifesto reads, in its entirety, as follows:

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value Individuals and interactions over processes and tools

- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan
- That is, while there is value in the items on the right, we value the items on the left more.

The meanings of the Manifesto items on the left within the agile software development context are described below.

Individuals and Interactions – in agile development, self-organization and motivation are important, as are interactions like co-location and pair programming. Working software – working software will be more useful and welcome than just presenting documents to clients in meetings. Customer collaboration – requirements cannot be fully collected at the beginning of the software development cycle, therefore continuous customer or stakeholder involvement is very important. Responding to change – agile development is focused on quick responses to change and continuous development.

Twelve principles underlie the Agile Manifesto, including:

- Customer satisfaction by rapid delivery of useful software
- Welcome changing requirements, even late in development
- Working software is delivered frequently (weeks rather than months)
- Working software is the principal measure of progress
- Sustainable development, able to maintain a constant pace
- Close, daily co-operation between business people and developers
- Face-to-face conversation is the best form of communication (co-location)
- Projects are built around motivated individuals, who should be trusted
- Continuous attention to technical excellence and good design
- Simplicity
- Self-organizing teams
- Regular adaptation to changing circumstances

In 2005, a group headed by Alistair Cockburn and Jim Highsmith wrote an addendum of project management principles, the Declaration of Interdependence, to guide software project management according to agile development methods.

CHARACTERISTICS

4.AGILE SOFTWARE DEVELOPMENT CHARACTERISTICS

There are many specific agile development methods. Most promote development, teamwork, collaboration, and process adaptability throughout the life-cycle of the project. Agile methods break tasks into small increments with minimal planning, and do not directly involve long-term planning. Iterations are short time frames (timeboxes) that typically last from one to four weeks. Each iteration involves a team working through a full software development cycle including planning, requirements analysis, design, coding, unit testing, and acceptance testing when a working product is demonstrated to stakeholders. This minimizes overall risk and allows the project to adapt to changes quickly. Stakeholders produce documentation as required. An iteration may not add enough functionality to warrant a market release, but the goal is to have an available release (with minimal bugs) at the end of each iteration.Multiple iterations may be required to release a product or new features. Team composition in an agile project is usually cross-functional and self-organizing without consideration for any existing corporate hierarchy or the corporate roles of team members. Team members normally take responsibility for tasks that deliver the functionality an iteration requires. They decide individually how to meet an iteration's requirements. Agile methods emphasize face-to-face communication over written documents when the team is all in the same location. Most agile teams work in a single open office (called a bullpen), which facilitates such communication. Team size is typically small (5-9 people) to simplify team communication and team collaboration. Larger development efforts may be delivered by multiple teams working toward a common goal or on different parts of an effort. This may require a co-ordination of priorities across teams. When a team works in different locations, they maintain daily contact through videoconferencing, voice, e-mail, etc. No matter what development disciplines are required, each agile team will contain a customer representative. This person is appointed by stakeholders to act on their behalf and makes a personal commitment to being available for developers to answer mid-iteration problem-domain questions. At the end of each iteration, stakeholders and the customer representative review progress and re-evaluate priorities with a view to optimizing the return on investment (ROI) and ensuring alignment with customer needs and company goals.

Most agile implementations use a routine and formal daily face-to-face communication among team members. This specifically includes the customer representative and any interested stakeholders as observers. In a brief session, team members report to each other what they did the previous day, what they intend to do today, and what their roadblocks are. This face-to-face communication exposes problems as they arise.

Agile development emphasizes working software as the primary measure of progress. This, combined with the preference for face-to-face communication, produces less written documentation than other methods. The agile method encourages stakeholders to prioritize wants with other iteration outcomes based exclusively on business value perceived at the beginning of the iteration (also known as value-driven).

Specific tools and techniques such as continuous integration, automated or xUnit test, pair programming, test driven development, design patterns, domain-driven design, code refactoring and other techniques are often used to improve quality and enhance project agility.

TRADITIONAL SOFTWARE DEVELOPMENT

5.TRADITIONAL SOFTWARE DEVELOPMENT

What follows is a description of the traditional/waterfall lifecycle, which agile is not:

Requirements

The first step in the Traditional Software Development Process is to identify requirements as well as the scope of the release. It encompasses those tasks that go into determining the needs or conditions to meet for a new or altered product, taking account of the possibly conflicting requirements of the various stakeholders, such as beneficiaries or users.

Architecture and Design

The goal of the architecture and design phase is to try to identify an architecture that has a good chance of working. The architecture is often defined using free-form diagrams which explore the technical infrastructure, and the major business entities and their relationships. The design is derived in a modeling session, in which issues are explored, until the team is satisfied that they understand what needs to be delivered.

Development

The development phase produces code in an environment that is specific to the culture of the development team and the skills of the individuals. In large projects the tasks are structured and distributed to teams. In small projects, or within smaller teams, the tasks are distributed according to team culture and skills. Development continues until goals or milestones are reached.

Testing, delivery, and feedback

Testing is ongoing at the local level and becomes more structured on larger scales and as the project approaches delivery. In large projects the testing is formalized. The customer is engaged in testing and feedback cycles when the development teams are relatively convinced that the software meets the requirements.

One of the similarities of the Agile Methodology and the traditional method is to conduct the testing of the software as it is being developed. The unit testing is achieved from the developer's perspective and the acceptance testing is conducted from the customer's perspective. the key difference in this regard is that in the Agile method the customer and developers are in close communication, where as in the traditional method, the "customer" is initially represented by the requirement and design documents.

COMPARISON WITH OTHER METHODS

6.COMPARISON WITH OTHER METHORDS

Agile methods are sometimes characterized as being at the opposite end of the spectrum from plan-driven or disciplined methods; agile teams may, however, employ highly disciplined formal methods. A more accurate distinction is that methods exist on a continuum from adaptive to predictive. Agile methods lie on the adaptive side of this continuum. Adaptive methods focus on adapting quickly to changing realities. When the needs of a project change, an adaptive team changes as well. An adaptive team will have difficulty describing exactly what will happen in the future. The further away a date is, the more vague an adaptive method will be about what will happen on that date. An adaptive team cannot report exactly what tasks are being done next week, but only which features are planned for next month. When asked about a release six months from now, an adaptive team may only be able to report the mission statement for the release, or a statement of expected value vs. cost.

Predictive methods, in contrast, focus on planning the future in detail. A predictive team can report exactly what features and tasks are planned for the entire length of the development process. Predictive teams have difficulty changing direction. The plan is typically optimized for the original destination and changing direction can require completed work to be started over. Predictive teams will often institute a change control board to ensure that only the most valuable changes are considered.

Formal methods, in contrast to adaptive and predictive methods, focus on computer science theory with a wide array of types of provers. A formal method attempts to prove the absence of errors with some level of determinism. Some formal methods are based on model checking and provide counterexamples for code that cannot be proven. Generally, mathematical models (often supported through special languages - see SPIN model checker) map to assertions about requirements.

Formal methods are dependent on a tool driven approach, and may be combined with other development approaches. Some provers do not easily scale. Like agile methods, manifestos relevant to high integrity software have been proposed in Crosstalk.

Agile methods have much in common with the Rapid Application Development techniques from the 1980/90s as espoused by James Martin and others.

AGILE METHORDS

7.AGILE METHORDS

Well-known ***agile software development methods*** include:

- **Agile Modeling**
- **Agile Unified Process (AUP)**
- **Dynamic Systems Development Method (DSDM)**
- **Essential Unified Process (EssUP)**
- **Extreme Programming (XP)**
- **Feature Driven Development (FDD)**
- **Open Unified Process (OpenUP)**
- **Scrum**
- **Velocity tracking**

AGILE SOFTWARE DEVELOPMENT TECHNIQUES

8.AGILE SOFTWARE DEVELOPMENT TECHNIQUES

Here are some of most popular agile software development techniques.

SCRUM

Scrum is an iterative, incremental framework for project management often seen in agile software development, a type of software engineering.

Although the Scrum approach was originally suggested for managing product development projects, its use has focused on the management of software development projects, and it can be used to run software maintenance teams or as a general project/program management approach.

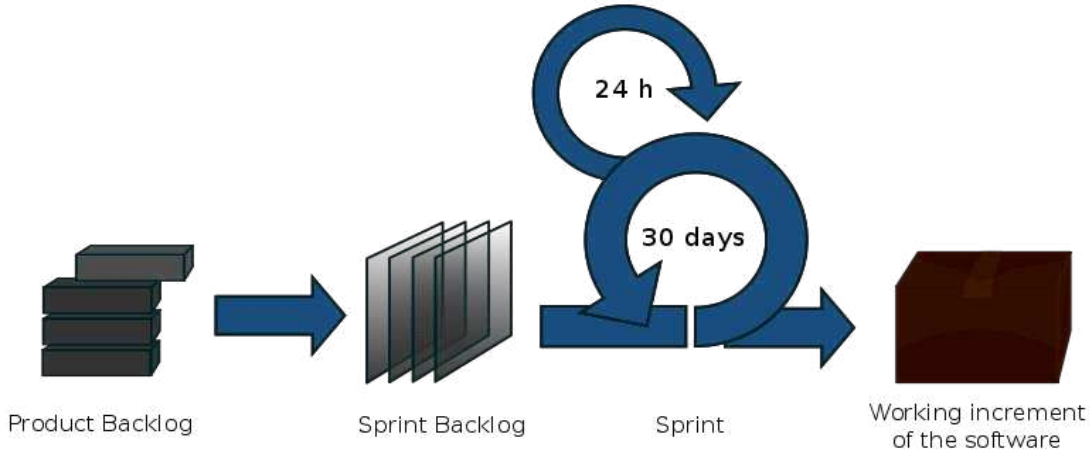

Fig:8.1 Scrum

Scrum is a process skeleton that contains sets of practices and predefined roles. The main roles in Scrum are:

1. The “ScrumMaster”, who maintains the processes
2. The “Product Owner”, who represents the stakeholders and the business
3. The “Team”, a cross-functional group who do the actual analysis, design, implementation, testing, etc.

During each “sprint”, typically a two to four week period (with the length being decided

by the team), the team creates a potentially deliverable product increment (for example, working and tested software). The set of features that go into a sprint come from the product "backlog", which is a prioritized set of high level requirements of work to be done. Which backlog items go into the sprint is determined during the sprint planning meeting. During this meeting, the Product Owner informs the team of the items in the product backlog that he or she wants completed. The team then determines how much of this they can commit to complete during the next sprint, and records this in the sprint backlog. During a sprint, no one is allowed to change the sprint backlog, which means that the requirements are frozen for that sprint. Development is timeboxed such that the sprint must end on time; if requirements are not completed for any reason they are left out and returned to the product backlog. After a sprint is completed, the team demonstrates how to use the software. Scrum enables the creation of self-organizing teams by encouraging co-location of all team members, and verbal communication between all team members and disciplines in the project.

A key principle of Scrum is its recognition that during a project the customers can change their minds about what they want and need (often called requirements churn), and that unpredicted challenges cannot be easily addressed in a traditional predictive or planned manner. As such, Scrum adopts an empirical approach—accepting that the problem cannot be fully understood or defined, focusing instead on maximizing the team's ability to deliver quickly and respond to emerging requirements.

Like other agile development methodologies, Scrum can be implemented through a wide range of tools. Many companies use universal software tools, such as spreadsheets to build and maintain artifacts such as the sprint backlog. There are also open-source and proprietary software packages dedicated to management of products under the Scrum process. Other organizations implement Scrum without the use of any software tools, and maintain their artifacts in hard-copy forms such as paper, whiteboards, and sticky notes.

Agile software development with scrum

Scrum has not only reinforced the interest in software project management, but also challenged the conventional ideas about such management. Scrum focuses on project management institutions where it is difficult to plan ahead with mechanisms for empirical process control, such as where feedback

loops constitute the core element of product development compared to traditional command-and-control oriented management. It represents a radically new approach for planning and managing software projects, bringing decision-making authority to the level of operation properties and certainties. Scrum reduces defects and makes the development process more efficient, as well as reducing long-term maintenance costs

EXTREME PROGRAMMING

Extreme Programming (XP) is a software development methodology which is intended to improve software quality and responsiveness to changing customer requirements. As a type of agile software development, it advocates frequent "releases" in short development cycles (timeboxing), which is intended to improve productivity and introduce checkpoints where new customer requirements can be adopted.

Other elements of extreme programming include: programming in pairs or doing extensive code review, unit testing of all code, avoiding programming of features until they are actually needed, a flat management structure, simplicity and clarity in code, expecting changes in the customer's requirements as time passes and the problem is better understood, and frequent communication with the customer and among programmers.The methodology takes its name from the idea that the beneficial elements of traditional software engineering practices are taken to "extreme" levels, on the theory that if some is good, more is better.

Critics have noted several potential drawbacks, including problems with unstable requirements, no documented compromises of user conflicts, and a lack of an overall design specification or document.

XP created quite a buzz in the late 1990s and early 2000s, seeing adoption in a number of environments radically different from its origins.The high discipline required by the original practices often went by the wayside, causing some of these practices, such as those thought too rigid, to be deprecated or reduced, or even left unfinished, on individual sites. For example, the practice of end-of-day integration tests, for a particular project, could be changed to an end-of-week schedule, or simply reduced to mutually agreed dates. Such a more relaxed schedule could avoid people feeling rushed to generate artificial stubs just to pass the end-of-day testing. A less rigid schedule allows, instead, for some complex features to be more fully developed over a several-day period. However, some level of periodic integration testing can detect groups of people working in non-compatible, tangent efforts before too much work is invested in divergent, wrong directions.

Meanwhile, other agile development practices have not stood still, and XP is still evolving, assimilating more lessons from experiences in the field, to use other practices. In the second edition of *Extreme Programming Explained*, Beck added more values and practices and differentiated between primary and corollary practices.

Feature Driven Development (FDD)

Feature Driven Development (FDD) is an iterative and incremental software development process. It is one of a number of Agile methods for developing software and forms part of the Agile Alliance. FDD blends a number of industry-recognized best practices into a cohesive whole. These practices are all driven from a client-valued functionality (feature) perspective. Its main purpose is to deliver tangible, working software repeatedly in a timely manner.

FDD is a model-driven short-iteration process that consists of five basic activities. For accurate state reporting and keeping track of the software development project, milestones that mark the progress made on each feature are defined. This section gives a high level overview of the activities.

Activities

FDD describes five basic activities that are within the software development process. In the figure on the right the meta-process model for these activities is displayed. During the first three sequential activities an overall model shape is established. The final two activities are iterated for each feature (derived from the process description in the ´Article´ section of Jeff De Luca´s website). The concepts involved in these activities are explained.

Develop overall model

The project starts with a high-level walkthrough of the scope of the system and its context. Next, detailed domain walkthroughs are held for each modeling area. In support of each domain, walkthrough models are then composed by small groups which are presented for peer review and discussion. One of the proposed models or a merge of them is selected which becomes the model for that particular domain area. Domain area models are merged into an overall model, the overall model shape being adjusted along the way.

Build feature list

The knowledge that is gathered during the initial modeling is used to identify a list of features. This is done by functionally decomposing the domain into subject areas. Subject areas each contain business activities, the steps within each business activity form the categorized feature list. Features

in this respect are small pieces of client-valued functions expressed in the form <action> <result> <object>, for example: ´Calculate the total of a sale´ or ´Validate the password of a user´. Features should not take more than two weeks to complete, else they should be broken down into smaller pieces.

Plan by feature

Now that the feature list is complete, the next step is to produce the development plan. Class ownership is done by ordering and assigning features (or feature sets) as classes to chief programmers.

Design by feature

A design package is produced for each feature. A chief programmer selects a small group of features that are to be developed within two weeks. Together with the corresponding class owners, the chief programmer works out detailed sequence diagrams for each feature and refines the overall model. Next, the class and method prologues are written and finally a design inspection is held.

Build by feature

After a successful design inspection a per feature activity to produce a completed client-valued function (feature) is being produced. The class owners develop the actual code for their classes. After a unit test and a successful code inspection, the completed feature is promoted to the main build.

Milestones

Since features are small, completing a feature is a relatively small task. For accurate state reporting and keeping track of the software development project it is however important to mark the progress made on each feature. FDD therefore defines six milestones per feature that are to be completed sequentially. The first three milestones are completed during the Design By Feature activity, the last three are completed during the Build By Feature activity. To help with tracking progress, a percentage complete is assigned to each milestone. In the table below the milestones (and their completion percentage) are shown. A feature that is still being coded is 44% complete (Domain Walkthrough 1%, Design 40% and Design Inspection 3% = 44%).

SOFTWARE DEVELOPMENT METHODOLOGIES

10.SOFTWARE DEVELOPMENT METHODOLOGIES

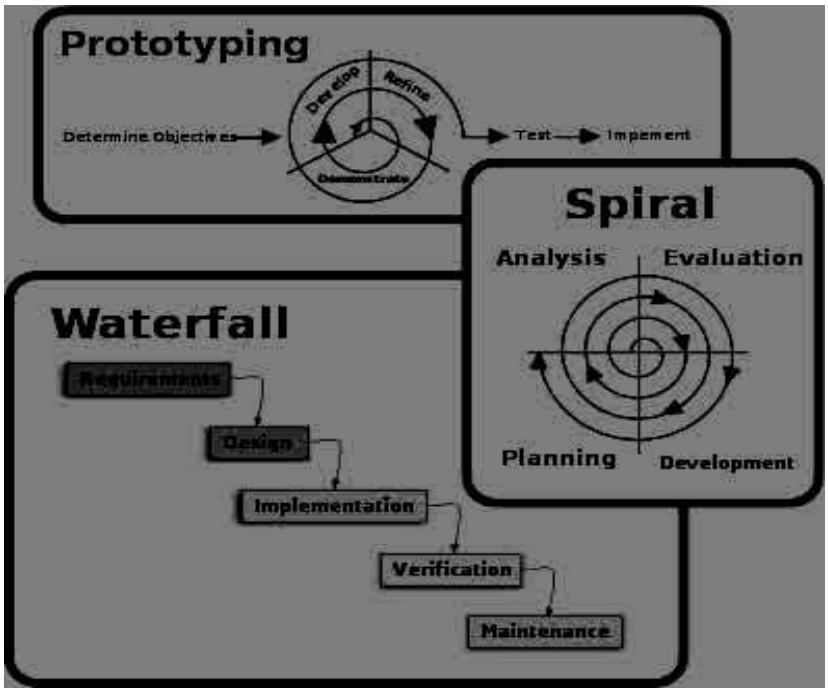

Fig:10.1 Prototyping

Software development methodology

A Software development methodology system development methodology in software engineering is a framework that is used to structure, plan, and control the process of developing an information system.

The software development methodology (also known as SDM) framework didn't emerge until the 1960s. According to Elliott (2004) the systems development life cycle (SDLC) can be considered to be the oldest formalized methodology framework for building information systems. The main idea of the SDLC has been "to pursue the development of information systems in a very deliberate, structured and methodical way, requiring each stage of the life cycle from inception of the idea to delivery of the final system, to be carried out rigidly and sequentially". within the context of the framework being applied. The main target of this methodology framework in the 1960s was "to develop large scale functional business systems in an age of large scale business conglomerates. Information systems activities revolved around heavy data processing and number crunching routines".

Every software development methodology framework acts as a basis for applying specific approaches to develop and maintain software. Several software development approaches have been used since the origin of information technology.

These are:

- **Waterfall:** a linear framework
- **Prototyping:** an iterative framework
- **Incremental:** a combined linear-iterative framework
- **Spiral:** a combined linear-iterative framework
- **Rapid application development (RAD):** an iterative framework
- **Extreme Programming**

Waterfall development

The Waterfall model is a sequential development approach, in which development is seen as flowing steadily downwards (like a waterfall) through the phases of requirements analysis, design, implementation, testing (validation), integration, and maintenance. The first formal description of the method is often cited as an article published by Winston W. Royce in 1970 although Royce did not use the term "waterfall" in this article.

The basic principles are:

- Project is divided into sequential phases, with some overlap and splashback acceptable between phases.
- Emphasis is on planning, time schedules, target dates, budgets and implementation of an entire system at one time.
- Tight control is maintained over the life of the project via extensive written documentation, formal reviews, and approval/signoff by the user and information technology management occurring at the end of most phases before beginning the next phase.

Prototyping

Software prototyping, is the development approach of activities during software development,

the creation of prototypes, i.e., incomplete versions of the software program being developed.

The basic principles are:

- Not a standalone, complete development methodology, but rather an approach to handling selected parts of a larger, more traditional development methodology (i.e. incremental, spiral, or rapid application development (RAD)).
- Attempts to reduce inherent project risk by breaking a project into smaller segments and providing more ease-of-change during the development process.
- User is involved throughout the development process, which increases the likelihood of user acceptance of the final implementation.
- Small-scale mock-ups of the system are developed following an iterative modification process until the prototype evolves to meet the users' requirements.
- While most prototypes are developed with the expectation that they will be discarded, it is possible in some cases to evolve from prototype to working system.
- A basic understanding of the fundamental business problem is necessary to avoid solving the wrong problem.

Incremental development

Various methods are acceptable for combining linear and iterative systems development methodologies, with the primary objective of each being to reduce inherent project risk by breaking a project into smaller segments and providing more ease-of-change during the development process.The basic principles are:

- A series of mini-Waterfalls are performed, where all phases of the Waterfall are completed for a small part of a system, before proceeding to the next increment, or
- Overall requirements are defined before proceeding to evolutionary, mini-Waterfall development of individual increments of a system.
- The initial software concept, requirements analysis, and design of architecture and system core are defined via Waterfall, followed by iterative Prototyping, which culminates in installing the final prototype, a working system.

Spiral development

The spiral model is a software development process combining elements of both design and prototyping-in-stages, in an effort to combine advantages of top-down and bottom-up concepts. It is a meta-model, a model that can be used by other models.

The basic principles are:

- Focus is on risk assessment and on minimizing project risk by breaking a project into smaller segments and providing more ease-of-change during the development process, as well as providing the opportunity to evaluate risks and weigh consideration of project continuation throughout the life cycle.
- "Each cycle involves a progression through the same sequence of steps, for each part of the product and for each of its levels of elaboration, from an overall concept-of-operation document down to the coding of each individual program."
- Each trip around the spiral traverses four basic quadrants: determine objectives, alternatives, and constraints of the iteration; evaluate alternatives; Identify and resolve risks; develop and verify deliverables from the iteration; and plan the next iteration.

- Begin each cycle with an identification of stakeholders and their win conditions, and end each cycle with review and commitment.

Rapid application development

Rapid application development (RAD) is a software development methodology, which involves iterative development and the construction of prototypes. Rapid application development is a term originally used to describe a software development process introduced by James Martin in 1991.

The basic principles are:

- Key objective is for fast development and delivery of a high quality system at a relatively low investment cost.
- Attempts to reduce inherent project risk by breaking a project into smaller segments and providing more ease-of-change during the development process.
- Aims to produce high quality systems quickly, primarily via iterative Prototyping (at any stage of development), active user involvement, and computerized development tools. These tools may include Graphical User Interface (GUI) builders, Computer Aided Software Engineering (CASE) tools, Database Management Systems (DBMS), fourth-generation programming languages, code generators, and object-oriented techniques.
- Key emphasis is on fulfilling the business need, while technological or engineering excellence is of lesser importance.
- Project control involves prioritizing development and defining delivery deadlines or "timeboxes". If the project starts to slip, emphasis is on reducing requirements to fit the timebox, not in increasing the deadline.
- Generally includes joint application design (JAD), where users are intensely involved in system design, via consensus building in either structured workshops, or electronically facilitated interaction.
- Active user involvement is imperative.
- Iteratively produces production software, as opposed to a throwaway prototype.
- Produces documentation necessary to facilitate future development and maintenance.
- Standard systems analysis and design methods can be fitted into this framework.

Measuring agility

While agility can be seen as a means to an end, a number of approaches have been proposed to quantify agility. *Agility Index Measurements* (AIM) score projects against a number of agility factors to achieve a total. The similarly named *Agility Measurement Index*, scores developments against five dimensions of a software project (duration, risk, novelty, effort, and interaction). Other techniques are based on measurable goals. Another study using fuzzy mathematics has suggested that project velocity can be used as a metric of agility. There are agile self assessments to determine whether a team is using agile practices (Nokia test, Karlskrona test, 42 points test). While such approaches have been proposed to measure agility, the practical application of such metrics has yet to be seen.

SUITABILITY

11.SUITABILITY

Large-scale agile software development remains an active research area. Agile development has been widely documented as working well for small (<10 developers) co-located teams. Some things that may negatively impact the success of an agile project are:

- Large-scale development efforts (>20 developers), through scaling strategies and evidence of some large projects have been described.
- Distributed development efforts (non-colocated teams). Strategies have been described in Bridging the Distance and Using an Agile Software Process with Offshore Developmen.t
- Forcing an agile process on a development team
- Mission-critical systems where failure is not an option at any cost (e.g. software for surgical procedures).

Several successful large-scale agile projects have been documented. BT has had several hundred developers situated in the UK, Ireland and India working collaboratively on projects and using Agile methods.

In terms of outsourcing agile development, Michael Hackett, Sr. Vice President of LogiGear Corporation has stated that "the offshore team should have expertise, experience, good communication skills, inter-cultural understanding, trust and understanding between members and groups and with each other."

Risk analysis can also be used to choose between adaptive (agile or value-driven) and predictive (plan-driven) methods.. Barry Boehm and Richard Turner suggest that each side of the continuum has its own home ground, as follows:

Agile home ground

- Low criticality
- Senior developers
- Requirements change often
- Small number of developers
- Culture that thrives on chaos

Plan-driven home ground

- High criticality
- Junior developers
- Requirements do not change often
- Large number of developer
- Culture that demands order

Formal methods

- Extreme criticality
- Senior developers
- Limited requirements, limited features see Wirth's law
- Requirements that can be modeled
- Extreme quality

Experience reports

Agile development has been the subject of several conferences. Some of these conferences have had academic backing and included peer-reviewed papers, including a peer-reviewed experience report track. The experience reports share industry experiences with agile software development.

As of 2010, experience reports have been or will be presented at the following conferences:

- XP (2000, 2001, 2002, 2003, 2004, 2005, 2006, 2010 (proceedings published by IEEE)
- XP Universe (2001)

- XP/Agile Universe (2002,2003,2004)
- Agile Development Conference(2003,2004,2007,2008) (proceedings published by IEEE)

APPLICATIONS

12. APPLICATIONS OF AGILLE WEB DEVELOPMENT

- Most of the new generation software companies and developers use agile software development as their strategy. (Eg: IBM)
- Diaspora, an opensource social networking site is developed in agile.
- Project SILPA, a collection of free indian language processing applications also folows agile web development techniques.
- Government institutions can adapt this method for software development.
- As there is a wide community support for opensource this may in turn help agile methods to be implemented.
- Agile software development is flexible method, its modifications for different project development and planning are available.
- The NSS-MCET BLOOD BANK Project, follows agile techniques to keep the software up to date with the current scenario.

REFERENCES

13.REFERENCES

[1] A b c Beck, Kent; et al. (2001). "Manifesto for Agile Software Development".

[2] Agile Alliance. http://agilemanifesto.org/. Retrieved 2010-06-14.

[3] A b Gerald M. Weinberg, as quoted in Larman, Craig; Basili, Victor R. (June 2003). "Iterative and Incremental Development: A Brief History".

[4] Edmonds, E. A. (1974). "A Process for the Development of Software for Nontechnical Users as an Adaptive System". General Systems 19: 215–18.

[5] Larman, Craig (2004). Agile and Iterative Development: A Manager's Guide. Addison-Wesley. p.27. ISBN 978-0-13-111155-4

[6] Ambler, S.W. "Examining the Agile Manifesto".

[7] Beck, Kent; et al. (2001). "Principles behind the Agile Manifesto".

[8] Agile Alliance. http://www.agilemanifesto.org/principles.html

[9] Anderson, David (2005). "Declaration of Interdependence". http://pmdoi.org.

[10] Beck, Kent (1999). "Embracing Change with Extreme Programming".

[11] A b Sliger, Michele; Broderick, Stacia (2008). The Software Project Manager's Bridge to Agility. Addison-Wesley. p.46. ISBN 0321502752.

[12] Black, S. E.; Boca., P. P.; Bowen, J. P.; Gorman, J.; Hinchey, M. G. (September 2009). "Formal versus agile: Survival of the fittest". IEEE Computer 49 (9): 39–45.

[13] Boehm, B.; R. Turner (2004). Balancing Agility and Discipline: A Guide for the Perplexed. Boston, MA: Addison-Wesley. ISBN 0-321-18612-5. Appendix A, pages 165-194

[14] A b Aydin, M.N., Harmsen, F., Slooten, K. v., & Stagwee, R. A. (2004). An Agile Information Systems Development Method in use. Turk J Elec Engin, 12(2), 127-138

[15] Abrahamsson, P., Warsta, J., Siponen, M.T., & Ronkainen, J. (2003). New Directions on Agile Methods: A Comparative Analysis. Proceedings of ICSE'03, 244-254

[16] A b c Abrahamsson, P., Salo, O., Ronkainen, J., & Warsta, J. (2002). Agile Software Development Methods: Review and Analysis. VTT Publications 478 A b Aydin, M.N., Harmsen, F., Slooten van K., & Stegwee, R.A. (2005). On the Adaptation of An Agile Information(Suren) Systems Development Method. Journal of Database Management Special issue on Agile Analysis, Design, and Implementation, 16(4), 20-24

[17] Mark Seuffert, Piratson Technologies, Sweden. "Karlskrona test, A generic agile adoption test". Piratson.se. http://www.piratson.se/archive/Agile_Karlskrona_Test.html. "How agile are you, A Scrum specific test". Agile-software-development.com. http://www.agile-software-development.com/2008/01/how-agile-are-you-take-this-42-point.html.

[18] "Answering the "Where is the Proof That Agile Methods Work" Question". Agilemodeling.com. 2007-01-19. http://www.agilemodeling.com/essays/proof.htm.

[19] "Agile Development Conference". Agile200x.org. http://www.agile200x.org/.